



ARA0110 PADRÕES DE PROJETO

Aula 12: Especificações Java - Parte 1





PLATAFORMA





PLATAFORMA

Java é mais do que uma linguagem de programação: é uma plataforma de computação completa formada por um ambiente de execução e um kit de desenvolvimento. Seu principal diferencial se torna a independência de plataforma, permitindo que códigos compilados executem em qualquer sistema operacional através da Máquina Virtual Java.



PLATAFORMA

A arquitetura do Java se baseia em compilar o código-fonte da aplicação (arquivos .java) em uma linguagem intermediária chamada **bytecode** (arquivos .class). Esse bytecode é então lido e executado pela JVM de cada sistema operacional. Isso nos garante a independência de plataforma, que é o objetivo final do Java.

LEMA

"Write Once, Run Anywhere"

"Escreva uma vez, rode em qualquer lugar"

COMPONENTES

JVM

Java Virtual Machine

JRE

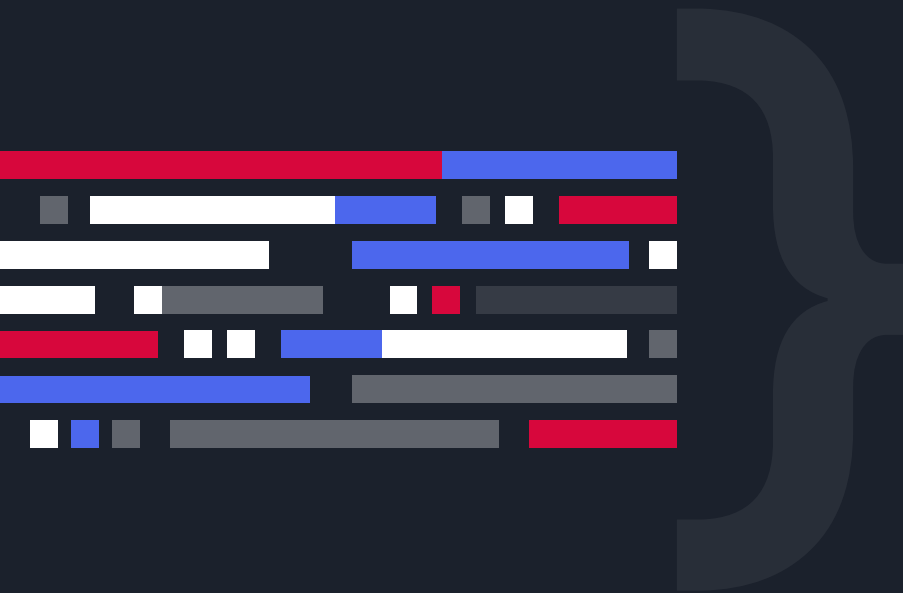
Java Runtime
Environment

JDK

Java Development
Kit

COMPONENTES

- **JVM (Java Virtual Machine):** A máquina virtual que executa os arquivos bytecode
- **JRE (Java Runtime Environment):** O ambiente que junta a JVM com as bibliotecas principais do Java, necessário apenas para executar programas
- **JDK (Java Development Kit):** O kit completo com ferramentas (como o compilador) para criar e compilar novas aplicações



PLATAFORMA

Além disso, a plataforma Java oferece suporte a várias linguagens de programação, como Kotlin, Scala e Groovy, que também são compiladas para bytecode e executadas dentro da mesma JVM.



EDIÇÕES DA PLATAFORMA

A plataforma expandiu-se para além do desenvolvimento de softwares básicos, segmentando-se em quatro edições principais. Cada uma atende a diferentes necessidades do mercado, desde sensores industriais até sistemas bancários:

- JSE - Java Standard Edition
- JEE - Jakarta Enterprise Edition
- JME - Java Micro Edition
- JFX - JavaFX



JSE

A Java Standard Edition é o núcleo da plataforma e a base para todas as outras edições. Nela, temos as APIs fundamentais, os tipos de dados básicos, estruturas de controle e as bibliotecas essenciais da linguagem (pacote `java.*`).



JSE

- **Público-alvo:** Desenvolvedores de software em geral, ferramentas de desktop e aplicações de console
- **Componentes-chave:** Inclui o JDK, a JVM e as bibliotecas essenciais (**java.lang**, **java.util**, **java.io**, **java.math** etc)
- **Recursos inclusos:** Segurança de rede, conectividade com bancos de dados (JDBC) e manipulação de arquivos (NIO2)
- **Uso prático:** Criação de ferramentas de linha de comando, scripts de automação e aplicações desktop tradicionais

EXEMPLO

```
public class Produto {  
    private String nome;  
    private Double preco;  
  
    public Produto(String nome, Double preco) {  
        this.nome = nome;  
        this.preco = preco;  
    }  
  
    @Override  
    public String toString() {  
        return String.format(  
            "Produto(nome=%s, preco=%.2f",  
            this.nome,  
            this.preco  
        );  
    }  
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
        Produto produto = new Produto(  
            "Notebook",  
            3_500.0  
        );  
        System.out.println(produto);  
    }  
}
```

JEE

Construída sobre a Java Standard Edition, a Jakarta Enterprise Edition (Antiga Java Enterprise Edition) adiciona um ecossistema de ferramentas para criar aplicações corporativas de grande escala, distribuídas, seguras e baseadas na web.

JEE

- **Mudança de nome:** A Oracle transferiu o controle para a Eclipse Foundation, mudando o nome de Java EE para Jakarta EE a partir da versão 8
- **Arquitetura:** Baseia-se em componentes modulares que rodam dentro de um servidor de aplicação
- **Principais especificações**
 - **Jakarta RESTful Web Services (JAX-RS):** Criação de APIs e serviços web
 - **Jakarta Persistence API (JPA):** Mapeamento de objetos de código para tabelas de bancos de dados (ORM)
 - **Jakarta Enterprise Beans (EJB):** Gerenciamento de lógicas de negócio, transações e segurança

CENÁRIOS DE USO

Sistemas bancários

E-commerce

Arquiteturas de
microsserviços

Portais
governamentais



JME

A Java Micro Edition se trata de uma versão enxuta projetada especificamente para dispositivos com recursos de hardware limitados, pouca memória, processamento fraco e baixa largura de banda. Em dispositivos com essas características, usar recursos de forma eficiente se torna um requisito obrigatório.



JME

- **Configurações e Perfis:** Adapta-se ao dispositivo usando Configurações (CLDC para dispositivos muito limitados) e Perfis (MIDP para celulares antigos)
- **Segurança:** Possui um modelo rígido de sandbox para garantir que aplicações não danifiquem o hardware ou acessem dados sensíveis
- **Uso prático:** Antigamente dominava os jogos e aplicativos de celulares “tijolão”. Hoje, foca em dispositivos de IoT, receptores de TV digital, smart cards (chips de cartão de crédito) e sensores industriais



JFX

Embora tenha nascido como parte do Java Standard Edition, o JavaFX tornou-se um projeto independente gerenciado pela comunidade OpenJFX, sendo focado na criação de interfaces gráficas modernas (GUI).

JFX

- **Substituto do Swing/AWT:** Veio para modernizar o antigo framework Swing, trazendo aceleração gráfica por hardware
- **Design flexível:** Permite separar a lógica do código da interface visual usando arquivos FXML (baseado em XML) e estilização CSS
- **Multimídia:** Suporta nativamente áudio, vídeo, gráficos 2D/3D e renderização de páginas web dentro da aplicação
- **Uso prático:** Dashboards corporativos detalhados, sistemas de quiosques multimídia, telas de caixas eletrônicos modernos e ferramentas de design

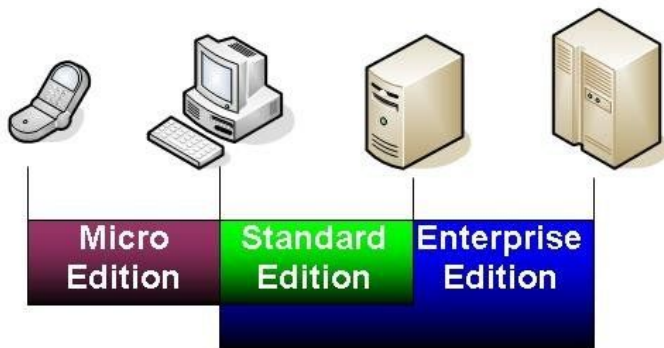


HORA DE EXERCÍCIO

Vamos criar uma aplicação de exemplo a ser desenvolvido durante a aula. Para criar esse projeto, vamos fazer uso do gerenciador de projetos Maven, e vamos usar um template base de projetos de aplicação web Java usando a especificação Jakarta EE.

PLATAFORMA JEE





DIFERENÇA

Quando comparamos JEE com JSE, a principal diferença entre as aplicações construídas nessas plataformas estão na arquitetura e no modelo de distribuição. Enquanto aplicações standalone são executadas diretamente no sistema operacional local, as aplicações web corporativas rodam em servidores e são acessadas via navegador, centralizando atualizações e dados.

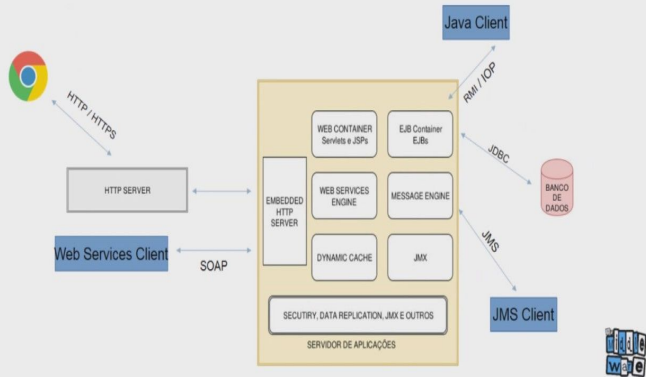
COMPARAÇÕES

Característica	Aplicações standalone	Aplicações web corporativas
Arquitetura	Processamento monolítico local. Roda diretamente sobre a JVM instalada	Cliente-servidor. A interface roda no navegador e a lógica de negócios no servidor
Distribuição e atualização	Requer instalação manual ou atualização por instaladores. Correções que exigem que o usuário baixe a nova versão	Centralizada no servidor. Qualquer alteração fica disponível imediatamente para todos os usuários
Acesso a recursos da máquina	Total. Acesso irrestrito a arquivos locais, hardware e periféricos	Limitado e intermediado pelo sandbox do navegador por questões de segurança

COMPARAÇÕES

Característica	Aplicações standalone	Aplicações web corporativas
Escalabilidade	Limitada aos recursos do computador onde a aplicação está em execução	Alta. Podem ser facilmente escaladas horizontalmente através de balanceadores de carga e contêineres de nuvem
Uso offline	Funciona perfeitamente sem conexão com a internet	Depende fundamentalmente de conexão com a internet ou de recursos locais complexos
Segurança	Depende das permissões do usuário no sistema operacional local. Acesso direto a bancos de dados é comum em redes internas	Centralizada, ideal para proteção de dados sensíveis. Bloqueia o acesso direto do cliente ao banco de dados, utilizando APIs controladas

Websphere Application Server



CONTAINER

Um servidor de aplicação da Jakarta EE atua como um ambiente de execução robusto que fornece infraestrutura e serviços padronizados (transações, segurança e persistência). Ele gerencia o ciclo de vida dos componentes, permitindo que os desenvolvedores foquem na lógica de negócio ao invés de codificar serviços de sistemas complexos.



SERVIDOR DE APLICAÇÃO

- **Gerenciamento de contêineres:** Hospeda e isola diferentes tipos de componentes, fornecendo serviços de injeção de dependências e gerenciamento de estado
- **Controle de Transações:** Garante a consistência dos dados de forma declarativa (através de anotações como **@Transactional**), controlando operações em bancos de dados sem a necessidade de controle manual
- **Segurança Integrada:** Implementa autenticação e autorização padronizadas, controlando o acesso a recursos e páginas de forma transparente





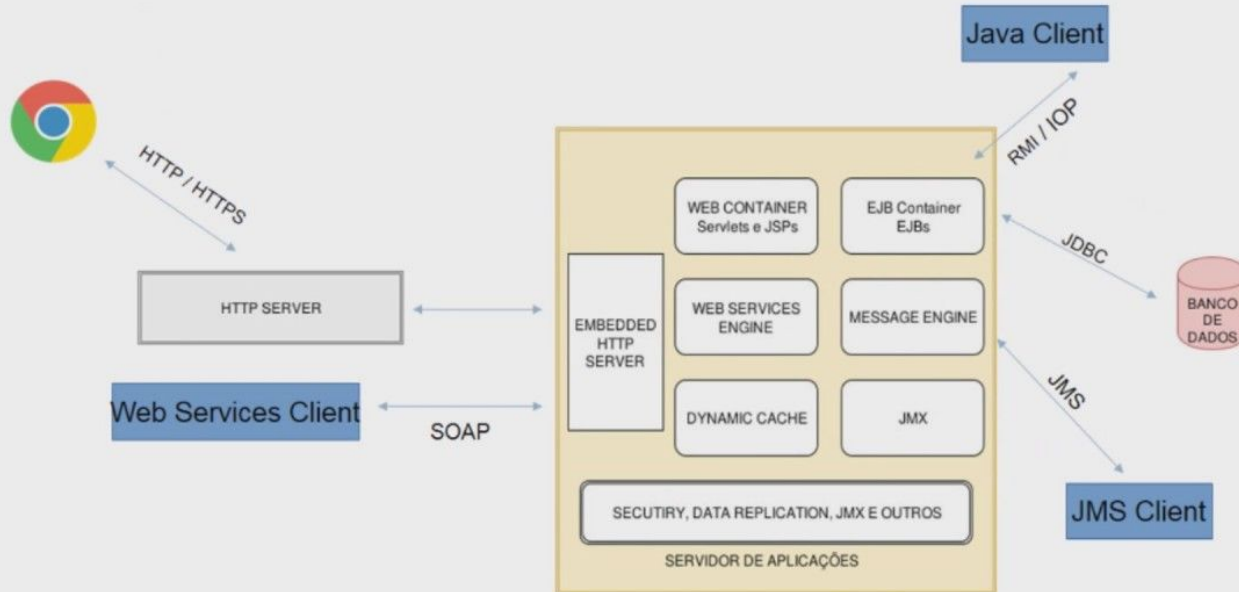
SERVIDOR DE APLICAÇÃO

- **Gerenciamento de Conexões e Recursos (pool):** Controla conexões com bancos de dados, filas de mensageria e outros recursos de forma otimizada para lidar com requisições simultâneas
- **Comunicação em Cluster:** Oferece suporte nativo à distribuição de carga (balanceador de carga) e tolerância a falhas em ambientes corporativos de alta disponibilidade



SERVIDOR DE APLICAÇÃO

Websphere Application Server



SERVIDORES DE CÓDIGO ABERTO



SERVIDORES PROPRIETÁRIOS



ORACLE®

The logo for Oracle WebLogic Server is presented within a light gray rectangular frame. It features the word "ORACLE" in a bold, red, sans-serif font, with a registered trademark symbol (®) to its upper right. A thin horizontal line is positioned directly beneath "ORACLE".

WebLogic Server

The words "WebLogic Server" are displayed in a black, serif font, centered below the horizontal line.



IBM

The IBM logo is centered at the top of a purple circular emblem. It consists of the letters "IBM" in a white, sans-serif font.

WebSphere

The word "WebSphere" is centered within a white rectangular border that has rounded corners. This border is superimposed on the purple circular background.

AS

The letters "AS" are centered at the bottom of the purple circular emblem in a white, sans-serif font.

ESPECIFICAÇÕES



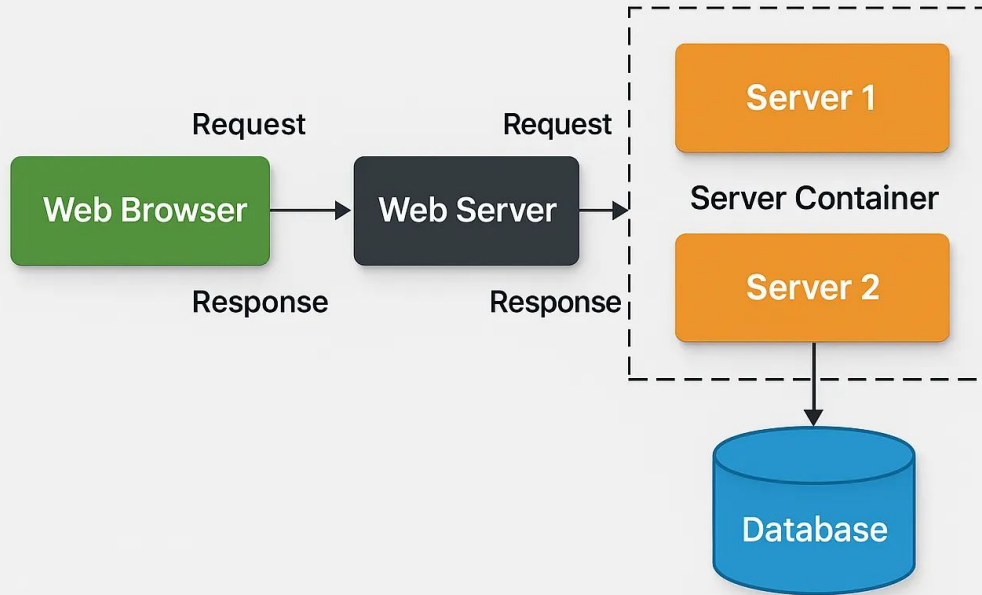


SERVLETS

Servlets são componentes Java que gerenciam requisições e respostas na web no Ecosystema JEE. Eles funcionam como uma camada intermediária entre o navegador web e o banco de dados ou lógica de negócios (semelhante a um controller de requisições). Os componentes desse módulo se encontram no pacote **`jakarta.servlet.*`**.

SERVLETS

Servlet Architecture



CICLO DE VIDA

Os servlets seguem um ciclo de vida estrito gerenciado pelo servidor de aplicação:

- **init()**: Executado apenas uma vez quando o servlet é carregado na memória
- **service()**: Direciona a requisição para o método HTTP correto
- **destroy()**: Executado uma vez antes do servlet ser removido da memória para liberar recursos

MÉTODOS HTTP

- **doGet:** Usado para buscar informações do servidor
- **doPost:** Usado para enviar ou salvar dados (como formulários)
- **doPut:** Usado para atualizar dados existentes
- **doDelete:** Usado para remover dados



HORA DE EXERCÍCIO

Vamos implementar um servlet responsável por gerenciar informações de produtos na nossa aplicação de exemplo.

COMANDO

```
mvn archetype:generate \  
  -DgroupId=br.estacio.amazonas \  
  -DartifactId=java-aula-padroes \  
  -DarchetypeArtifactId=maven-archetype-webapp \  
  -DinteractiveMode=false
```

DEPENDÊNCIA

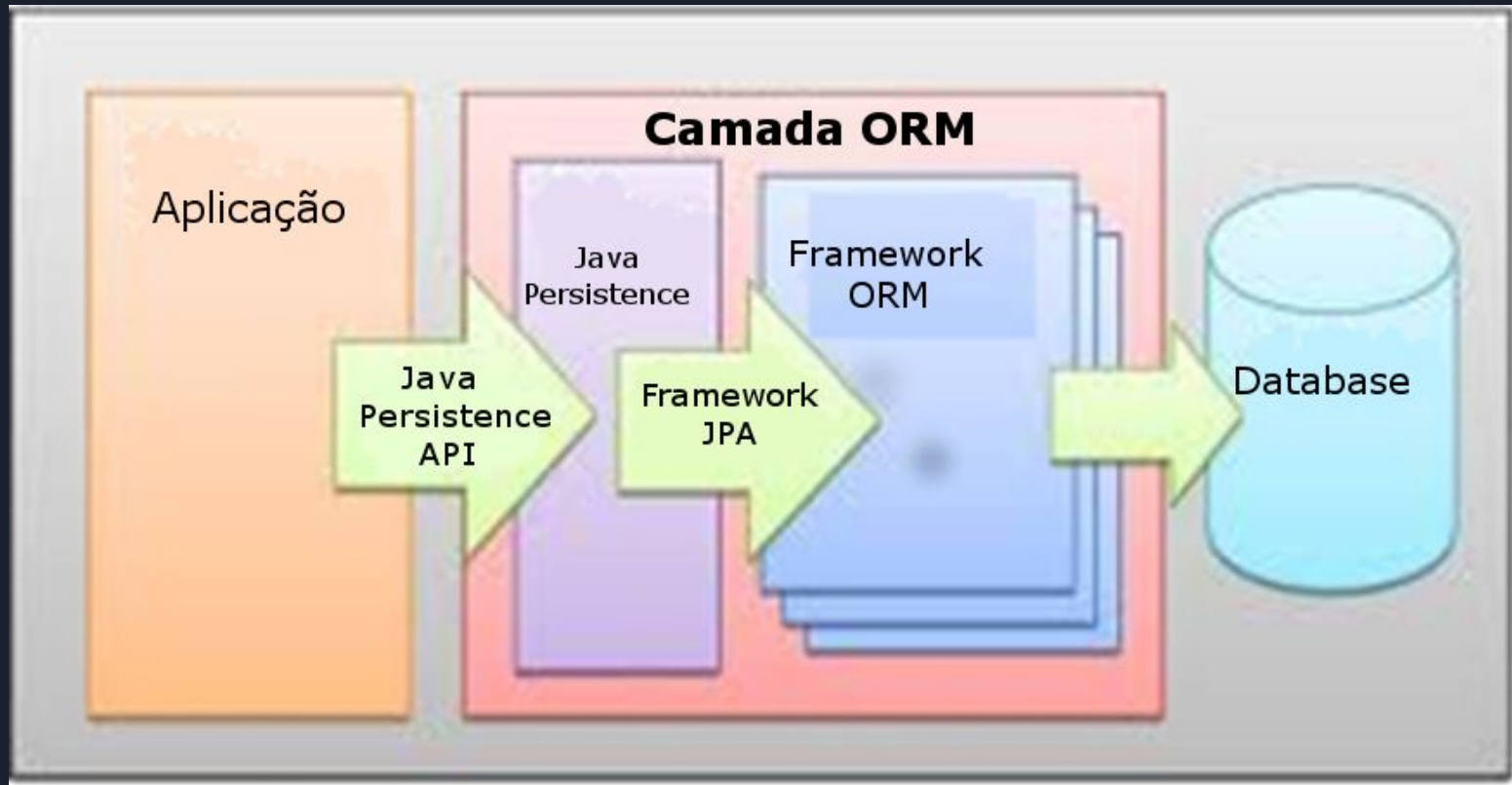
```
<dependency>  
  <groupId>jakarta.servlet</groupId>  
  <artifactId>jakarta.servlet-api</artifactId>  
  <version>6.0.0</version>  
  <scope>provided</scope>  
</dependency>
```



JPA

Jakarta Persistence API (JPA) é a especificação oficial do ecossistema para o gerenciamento de bancos de dados relacionais e mapeamento objeto-relacional (ORM). Ela funciona como uma ponte abstrata, permitindo que você manipule tabelas de bancos de dados utilizando apenas classes e objetos Java. Os componentes desse módulo se encontram no pacote **`jakarta.persistence.*`**.

JPA





PONTOS IMPORTANTES

- **Especificação x Framework:** A JPA define apenas as interfaces, regras e anotações. Ela não possui código executável próprio para rodar as consultas
- **Provedores de Persistência:** Para que a JPA funcione, você precisa de um framework que implementa essas regras. Os mais famosos são o Hibernate e o EclipseLink





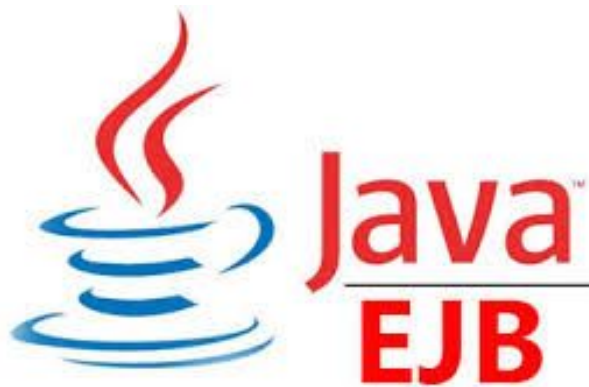
PILARES DA JPA

- **Entidades (@Entity):** Classes Java simples mapeadas diretamente para as tabelas do seu banco de dados utilizando anotações e metadados
- **Gerenciador de Entidades (EntityManager):** A API principal usada para gerenciar o ciclo de vida das entidades. É através dele que você insere (persist), atualiza (merge), remove (remove) ou busca (find) dados.
- **Unidade de Persistência (persistence.xml):** Arquivo de configuração onde você define as credenciais do banco de dados, o provedor ORM escolhido e as propriedades de conexão
- **Linguagem de Consulta:** Permite buscar dados utilizando objetos e propriedades em vez de tabelas. Suporta tanto a JPQL (Jakarta Persistence Query Language) quanto Criteria API (consultas dinâmicas e tipadas via código Java)



HORA DE EXERCÍCIO

Vamos fazer com que os dados de produtos, que são manipulados pela nossa aplicação, estejam persistidos em um banco de dados MySQL. A interação com o banco de dados será feita inteiramente com objetos Java, sem interação com código SQL.



EJB

Jakarta Enterprise Beans (EJB) é uma arquitetura de componentes do lado do servidor projetada para encapsular a lógica de negócios de aplicações corporativas robustas, escaláveis e seguras. Ao rodar dentro de um contêiner EJB, o desenvolvedor delega ao sistema tarefas complexas de infraestrutura, permitindo que ele foque exclusivamente nas regras de negócio da aplicação. Os componentes desse módulo se encontram nos pacotes **`jakarta.enterprise.concurrent.*`** e **`jakarta.ejb.*`**.



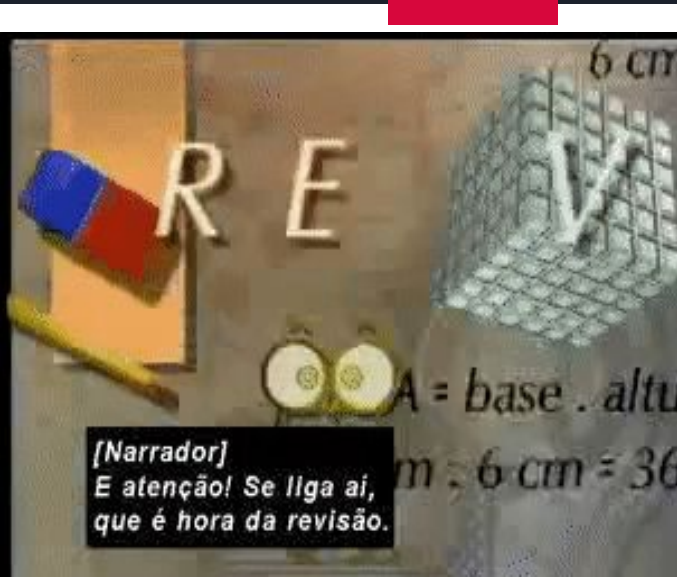
SERVIÇOS NATIVOS

- **Gerenciamento de Transações:** Controle declarativo e automático de commits e rollbacks de operações via JTA (Jakarta Transaction API)
 - **Segurança Integrada:** Restrição de acesso a métodos baseados em papéis (roles) de usuários de forma simples através de anotações
 - **Injeção de Dependências:** Integração transparente com recursos do servidor como bancos de dados (JPA) e outros serviços via CDI (Context and Dependency Injection)
 - **Acesso Remoto:** Capacidade de invocar lógica de negócios distribuída em outras máquinas ou servidores através da JNDI (Jakarta Naming and Directory Interface) e protocolos de rede
 - **Concorrência e Ciclo de Vida:** O contêiner gerencia threads, paralelismo e criação/destruição de instâncias automaticamente
- 



TIPOS DE COMPONENTES

- **Session Beans (processamento de lógica):**
 - **Stateless (@Stateless):** Não guardam o estado do cliente entre chamadas. Funcionam como serviços puros de alta performance mantidos em um pool pelo servidor
 - **Stateful (@Stateful):** Mantém o estado conversacional com um cliente específico ao longo de múltiplas interações
 - **Singleton (@Singleton):** Uma única instância compartilhada por toda a aplicação, ideal para caches ou rotinas de inicialização global
- **Message-Driven Beans (processamento assíncrono):**
 - **MDB (@MessageDriven):** Componentes que reagem estritamente ao recebimento de mensagens vindas de sistemas de mensageria. Processam dados de forma totalmente assíncrona, desacoplando os sistemas



RECAPITULANDO

- Java como plataforma
- Edições da plataforma
- Jakarta Enterprise Edition
- Servidores de aplicação
- Servlets, JPA e EJB

OBRIGADO!



"EU GOSTO DE DINHEIRO"
Seu Sirigueijo